



PRÁCTICA 3

Sincronización de Procesos Distribuidos – Aerolíneas Rafael Pabón

Objetivo General

Diseñar e implementar un sistema de reservas aéreas distribuido que garantice la **consistencia global del estado**, utilizando mecanismos de **sincronización de procesos distribuidos**, específicamente **relojes vectoriales y relojes de Lamport**, para evitar conflictos como la sobreventa de asientos.

Contexto del Problema

La empresa **Aerolíneas Rafael Pabón** opera vuelos a múltiples destinos alrededor del mundo y ha decidido implementar un **sistema de reservas en línea distribuido**.

El sistema está compuesto por **3 servidores distribuidos geográficamente**, encargados de procesar operaciones de:

- Reserva de boletos
- Venta de boletos
- Anulación (devolución) de boletos

Debido a la concurrencia de operaciones entre nodos, es fundamental mantener un **estado global consistente**, evitando inconsistencias como:

- Doble asignación de asientos
- Estados de vuelo incoherentes
- Problemas de sincronización entre bases de datos

Datos del Sistema

Aeropuertos disponibles

ATL (USA) PEK (CHN) DXB (ARE) TYO (JPN) LON (GBR)
LAX (USA) PAR (FRA) FRA (DEU) IST (TUR) SIN (SGP)

MAD (ESP) AMS (NLD) DFW (USA) CAN (CHN) SAO (BRA)

⚠ No todos los destinos tienen rutas de retorno.

Tipos de Asientos

- Primera Clase
- Clase Turística

Modelos de Aviones

Modelo	Primera Clase	Económica
Airbus A380-800	10	439
Boeing 777-300ER	10	300
Airbus A350-900	12	250
Boeing 787-9 Dreamliner	8	220

Flota disponible

- 6 × Airbus A380-800
- 18 × Boeing 777-300ER
- 11 × Airbus A350-900
- 15 × Boeing 787-9 Dreamliner

Estados de vuelo

SCHEDULED

BOARDING

DEPARTED

IN_FLIGHT

LANDED

ARRIVED

DELAYED

CANCELLED

Datos de Entrada

Se proporcionan:

1. Matriz de costos (Clase Turística)
2. Matriz de costos (Primera Clase)
3. Matriz de tiempos de vuelo

4. Dataset de **60,000 vuelos programados**:

<https://drive.google.com/file/d/1ZvH-IeC9aki8hNDMPfrGblvni6DYf/view?usp=sharing>

Reglas del Sistema

Actualización de estados

Antes de ejecutar cualquier procesamiento:

Todos los vuelos con fecha posterior a la actual deben actualizarse al estado **"SCHEDULED"**.

Revisar que todos los no aparezcan mágicamente en otros aeropuertos para iniciar una nueva ruta

Devoluciones y consistencia eventual

Cuando un asiento cambia de estado de **"REFUNDED"** a **"AVAILABLE"**:

- La propagación a los otros nodos puede tardar **15 minutos (configurable)**
- Esto puede generar **inconsistencias temporales** entre bases de datos
- Debe modelarse como un problema de **consistencia eventual**



Distribución de estados

- 73% de los asientos → **VENDIDOS (configurable)**
- 3% de los asientos → **RESERVADOS**

Requerimientos Técnicos

El sistema debe implementar:

◆ Sincronización distribuida

- Relojes de Lamport
- Relojes vectoriales

◆ Control de concurrencia

- Prevención de doble reserva
- Ordenamiento de eventos

◆ Algoritmos adicionales

- Algoritmo de **Dijkstra** o similares
- Resolución del problema del **Agente Viajero (TSP)** considerando:
 - Menor costo
 - Menor tiempo

Requerimientos Funcionales

El sistema debe permitir:

- Consultar vuelos disponibles
- Reservar asientos
- Comprar boletos
- Cancelar boletos
- Generar rutas óptimas

2 Dashboard (1 Del Vuelo y 2 Del estado de todos los vuelos)

Se debe implementar un panel que muestre:

- Total de asientos vendidos
- Total de reservas
- Disponibilidad por vuelo
- Ingresos por:
 - Primera clase
 - Clase turística

Experiencia de Usuario

El sistema debe:

- Tener flujo claro e intuitivo
- Soportar **multiidioma**
- Estar orientado a usuarios globales

Generación de Pasajes

- Generar al menos **boletos en PDF** o Los boletos deben ser compatibles con Aplicaciones tipo **Wallet (Passes & Tickets)**



Criterios de Evaluación (100 puntos)

Criterio	Puntaje
Algoritmos (Dijkstra / TSP)	30 pts
Experiencia de usuario (UX/UI, multiidioma)	30 pts
Dashboard de gerencial	20 pts
Sincronización de bases de datos	10 pts
Generación de pasajes (PDF + Wallet)	10 pts

Sincronización:

- 5 pts → 2 base de datos
- 10 pts → 3 bases de datos

Entregables

1. Código fuente del sistema
2. Documentación técnica:
 - Arquitectura distribuida
 - Estrategia de sincronización
3. Dashboard funcional
4. Archivos PDF de pasajes
5. Evidencia de funcionamiento

Nota Final

Esta práctica evalúa la capacidad de integrar:

- **Sistemas distribuidos**
- **Algoritmos clásicos**
- **Consistencia de datos**
- **Experiencia de usuario real**